

Algorithms

NAME:SIRI DONI

HALL TICKET NO: 2403A51075

LAB – 1 .

Bubble Sort C Code:

```
#include <stdio.h>

int main() {
    int n, i, j, temp;

    printf("Enter number of elements: ");

    scanf("%d", &n);

    int arr[n];

    printf("Enter elements:\n");

    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    for(i = 0; i < n - 1; i++) {
        for(j = 0; j < n - i - 1; j++) {
            if(arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

```

}

printf("Sorted array:\n");

for(i = 0; i < n; i++) {

    printf("%d ", arr[i]);

}

return 0;

}

```

```

C:\Users\Rasagna\OneDrive\...
Enter number of elements: 8
Enter elements:
2 54 67 9 3 11 71 87
Sorted array:
2 3 9 11 54 67 71 87
-----
Process exited after 41.64 seconds with return value 0
Press any key to continue . . .

```

Selection C Code:

```

#include <stdio.h>

int main() {

    int arr[] = {64, 25, 12, 22, 11};

    int n = 5;

    int i, j, min_index, temp;

    for (i = 0; i < n - 1; i++) {

        min_index = i;

        for (j = i + 1; j < n; j++) {

            if (arr[j] < arr[min_index]) {

                min_index = j;

            }

        }

    }

}

```

```

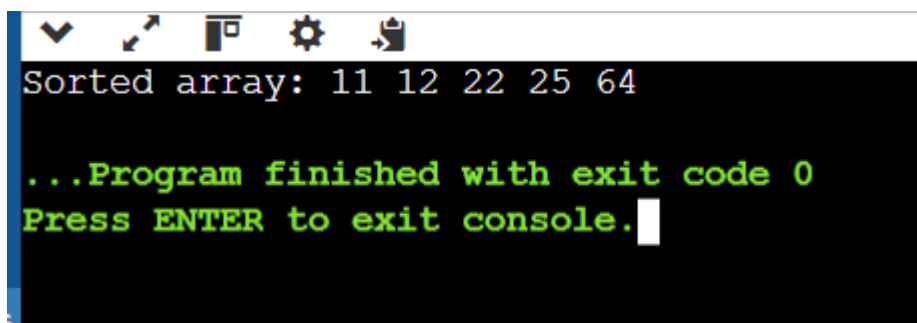
    }

    temp = arr[i];
    arr[i] = arr[min_index];
    arr[min_index] = temp;
}

printf("Sorted array: ");
for (i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}

return 0;
}

```



```

Sorted array: 11 12 22 25 64

...Program finished with exit code 0
Press ENTER to exit console.

```

Insertion sort C Code:

```

#include <stdio.h>

int main() {
    int n, i, j, key;

    printf("Enter number of elements: ");

    scanf("%d", &n);

    int arr[n];

    printf("Enter elements:\n");

    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
}

```

```

}
for(i = 1; i < n; i++) {
    key = arr[i];
    j = i - 1;
    while(j >= 0 && arr[j] > key) {
        arr[j + 1] = arr[j];
        j--;
    }
    arr[j + 1] = key;
}
printf("Sorted array:\n");
for(i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}
return 0;
}

```

```

C:\Users\Rasagna\OneDrive\I >
Enter number of elements: 8
Enter elements:
12 34 67 55 14 11 6 2
Sorted array:
2 6 11 12 14 34 55 67
-----
Process exited after 32.58 seconds with return v
Press any key to continue . . . |

```

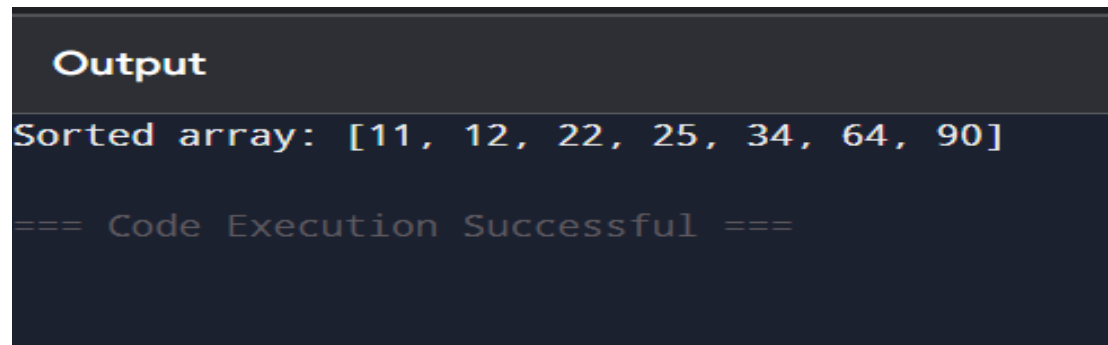
Bubble Sort Python Code:

```

arr = [64, 34, 25, 12, 22, 11, 90]
n = len(arr)

```

```
for i in range(n - 1):
    for j in range(n - i - 1):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]
print("Sorted array:", arr)
```

A screenshot of a code execution environment. At the top, the word "Output" is displayed in a bold, white font on a dark background. Below it, the text "Sorted array: [11, 12, 22, 25, 34, 64, 90]" is shown in a light blue monospace font. At the bottom, the text "=== Code Execution Successful ===" is displayed in a light green monospace font.

Output

Sorted array: [11, 12, 22, 25, 34, 64, 90]

=== Code Execution Successful ===

Selection Python code

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]
arr = [64, 25, 12, 22, 11]
selection_sort(arr)
print("Sorted array:", arr)
```

Output

Sorted array: [11, 12, 22, 25, 64]

=== Code Execution Successful ===

Insertion sort Python code:

```
arr = [64, 34, 25, 12, 22, 11, 90]
```

```
for i in range(1, len(arr)):
```

```
    key = arr[i]
```

```
    j = i - 1
```

```
    while j >= 0 and arr[j] > key:
```

```
        arr[j + 1] = arr[j]
```

```
        j -= 1
```

```
    arr[j + 1] = key
```

```
print("Sorted array:", arr)
```

Output

Sorted array: [11, 12, 22, 25, 34, 64, 90]

=== Code Execution Successful ===

Bubble Sort Java Code:

```
public class BubbleSort {
```

```
    public static void main(String[] args) {
```

```
        int[] arr = {64, 34, 25, 12, 22, 11, 90};
```

```
        for (int i = 0; i < arr.length - 1; i++) {
```

```

        for (int j = 0; j < arr.length - i - 1; j++) {

            if (arr[j] > arr[j + 1]) {

                int temp = arr[j];

                arr[j] = arr[j + 1];

                arr[j + 1] = temp;

            }

        }

    }

    System.out.println("Sorted array:");

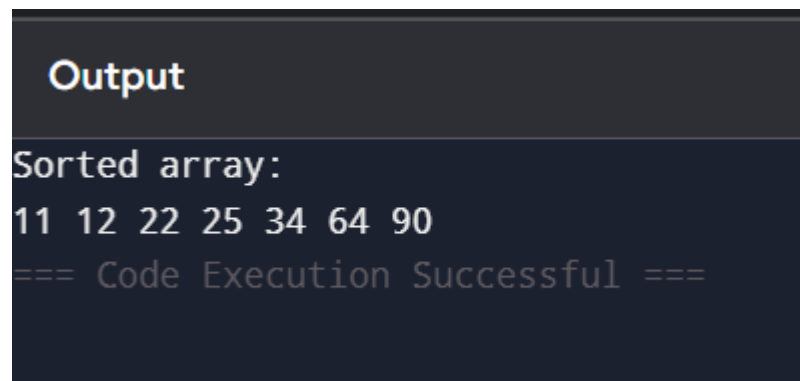
    for (int i = 0; i < arr.length; i++) {

        System.out.print(arr[i] + " ");

    }

}

```



The screenshot shows a dark-themed window titled "Output". Inside, the text "Sorted array:" is followed by the numbers "11 12 22 25 34 64 90" on the next line. Below that, the message "=== Code Execution Successful ===" is displayed in a lighter, monospaced font.

Insertion Sort Java Code:

```

public class InsertionSort {

    public static void main(String[] args) {

        int[] arr = {64, 34, 25, 12, 22, 11, 90};

        for (int i = 1; i < arr.length; i++) {

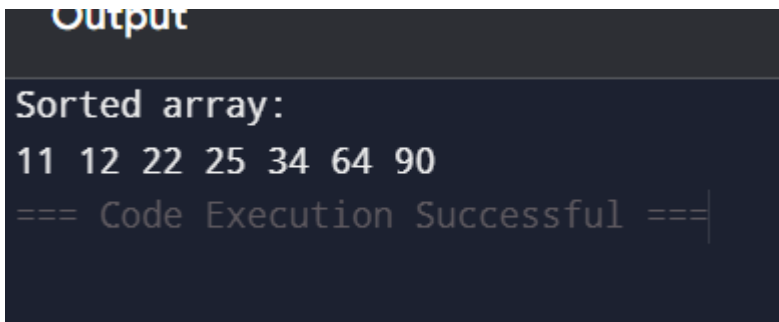
```

```

        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }

    System.out.println("Sorted array:");
    for (int i = 0; i < arr.length; i++) {
        System.out.print(arr[i] + " ");
    }
}
}

```



Output

```

Sorted array:
11 12 22 25 34 64 90
=== Code Execution Successful ===

```

Selection sort Java Code:

```

public class SelectionSort {
    public static void main(String[] args) {
        int[] arr = {64, 25, 12, 22, 11};
        for (int i = 0; i < arr.length - 1; i++) {
            int minIndex = i;
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[j] < arr[minIndex]) {

```



```
        minIndex = j;
    }
}
int temp = arr[i];
arr[i] = arr[minIndex];
arr[minIndex] = temp;
}
System.out.print("Sorted array: ");
for (int i = 0; i < arr.length; i++) {
    System.out.print(arr[i] + " ");
}
}
}
```

Output

```
Sorted array: 11 12 22 25 64
=== Code Execution Successful ===
```